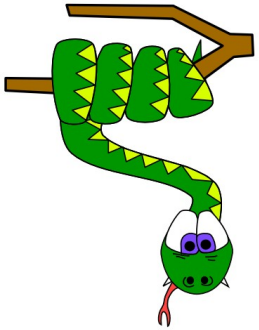


Inheritance

Inheritance

- Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a **child** or **derived class**) to inherit attributes and methods from another class (called a **parent** or **base class**).
- When a class is created via inheritance, it “inherits” the attributes defined by its base classes.
- However, a derived class may redefine any of these attributes and add new attributes of its own.

Inheritance

- Inheritance is specified with a comma-separated list of base-class names in the class statement.
- If there is no logical base class, a class inherits from object

Inheritance

- Inheritance is specified with a comma-separated list of base-class names in the class statement.
- If there is no logical base class, a class inherits from object

Python Inheritance Syntax

```
class BaseClass:
  Body of base class

class DerivedClass(BaseClass):
  Body of derived class
```

Inheritance

- Types of inheritance
  - Single inheritance
  - Multiple inheritance
  - Multi level inheritance
  - Hybrid inheritance
  - Hierarchical inheritance

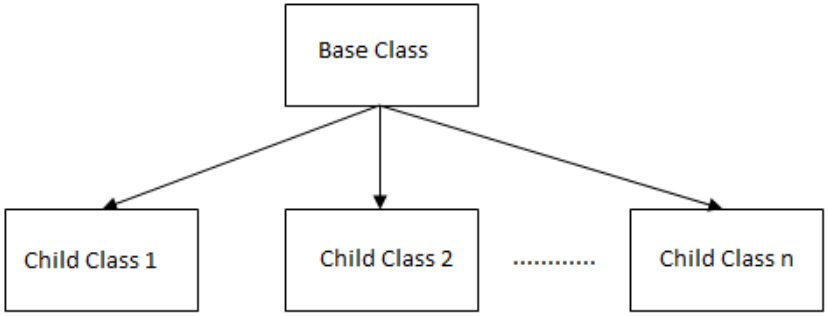
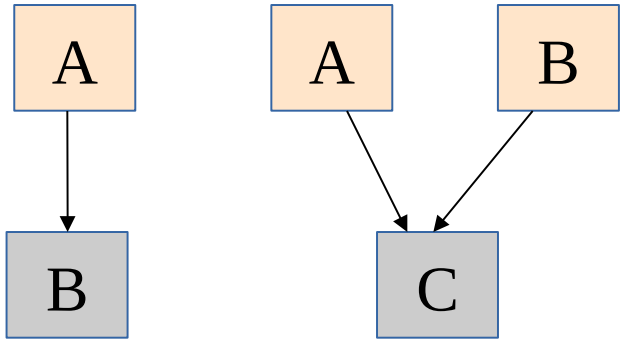
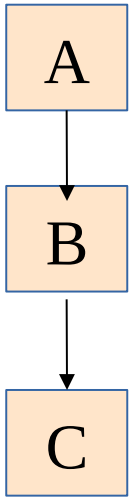
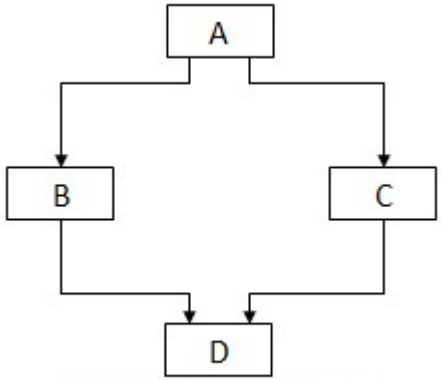


Fig: Hierarchical Inheritance



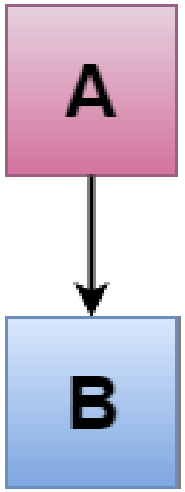




Hybrid Inheritance

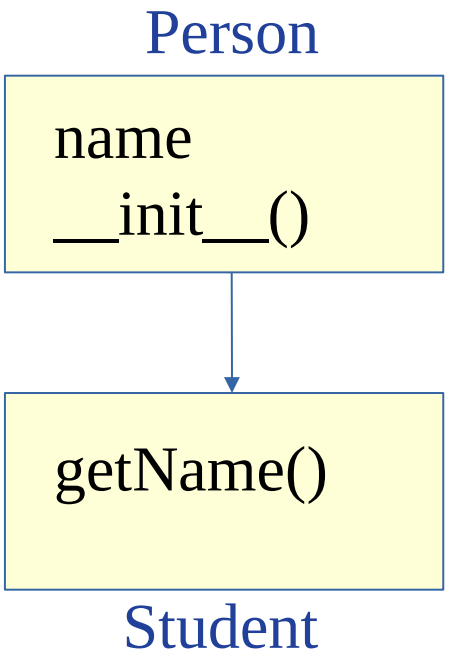
Single Inheritance

- Single inheritance enables a derived class to inherit properties and behavior from a single parent class.
- It allows a derived class to inherit the properties and behavior of a base class, thus enabling code reusability as well as adding new features to the existing code.



Single Inheritance

```
class Person:
  def __init__(self,x):
    self.name=x

class Student(Person):
  def getName(self):
    print(self.name)

ob=Student("Raj")
ob.getName()
```



Single Inheritance

__init__( ) Function

- The __init__() function is called every time a class is being used to make an object.
- When we add the __init__() function in a parent class, the child class will no longer be able to inherit the parent class's __init__() function.
- The child's class __init__() function **overrides** the parent class's __init__() function.
- The child class can call its parent class constructor by explicitly calling it like **ParentClass.__init__(self)**

Single Inheritance

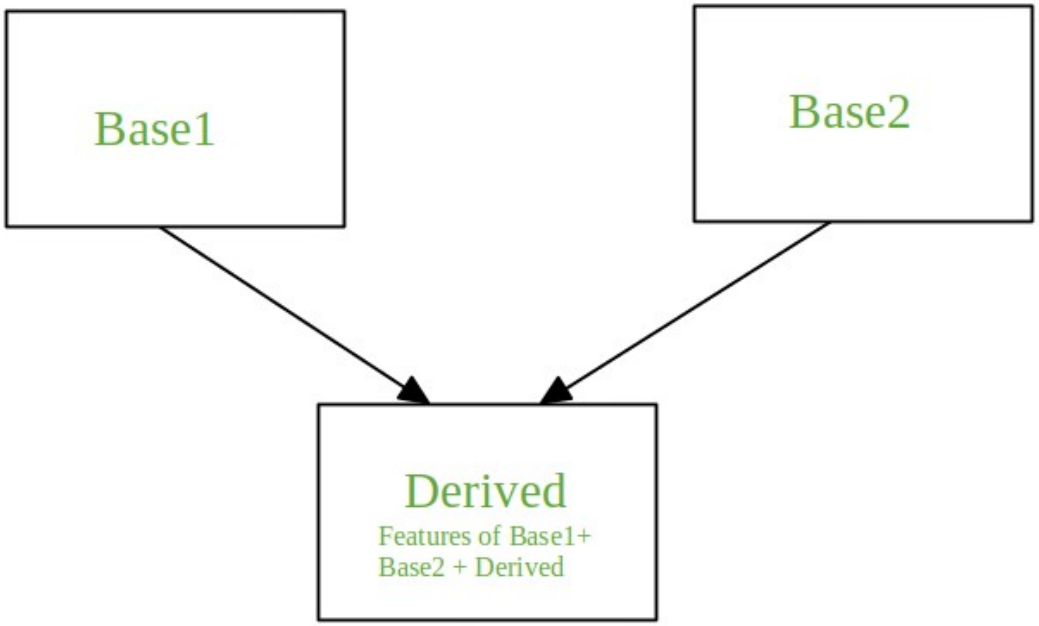
```
class Person:
    def __init__(self , fname):
        self.firstname = fname

    def view(self):
        print(self.firstname )
```

```
class Student(Person):
    def __init__(self , fname , no):
        Person.__init__(self, fname)
        self.roll_no=no

    def view(self):
        print("name=" , self.firstname ,
       "age=",  self.roll_no )
```

```
ob = Student("Ashok" , 28)
ob.view()
```

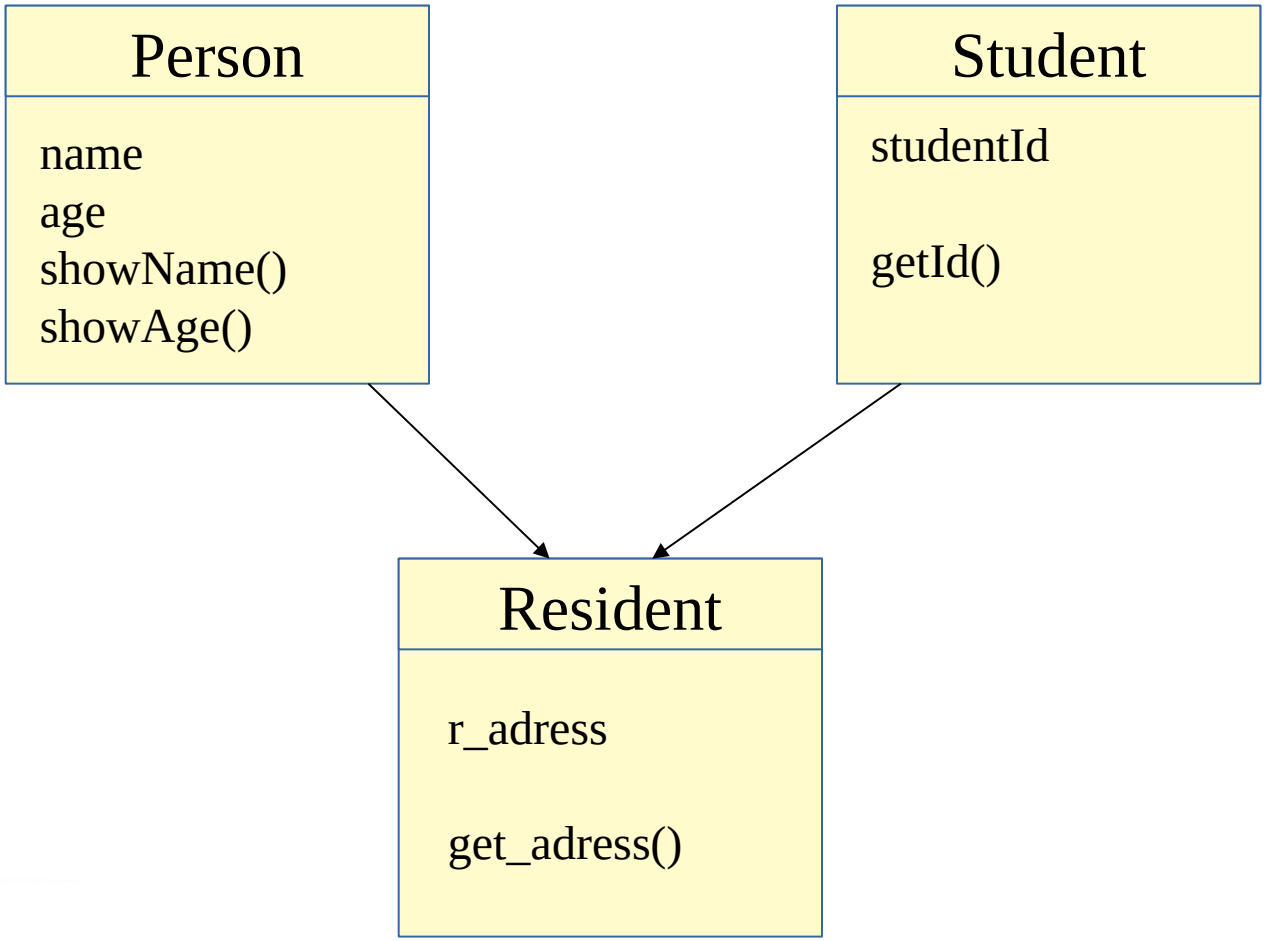
Multiple Inheritance

When a class is derived from more than one base class it is called multiple Inheritance. The derived class inherits all the features of the base case.



Multiple Inheritance

Example



```
class Person:
    #defining constructor
    def __init__(self, personName, personAge):
        self.name = personName
        self.age = personAge

    #defining class methods
    def showName(self):
        print(self.name)

    def showAge(self):
        print(self.age)

    #end of class definition
```

```
# defining another class
class Student:
    def __init__(self, studentId):
        self.studentId = studentId

    def getId(self):
        return self.studentId
```

```
class Resident(Person, Student): # extends both
Person and Student class
    def __init__(self, name, age, id,address):
        self.r_adress= address
        Person.__init__(self, name, age)
        Student.__init__(self, id)

    def get_address(self):
        return self.r_adress
```

```
# Create an object of the subclass
resident1 = Resident('John', 30, 'S101',’KANNUR’)
resident1.showName()
print(resident1.getId())
print(resident1.get_address())
```

Resolving the Conflicts with python multiple inheritance

Class C inherits both A and B. And both of them has an attribute ‘name’. From which class the value of name will be inherited in C?

```
class A:
    def __init__(self):
        self.name = 'John'
        self.age = 23

    def getName(self):
        return self.name


class B:
    def __init__(self):
        self.name = 'Richard'
        self.id = '32'

    def getName(self):
        return self.name
```

```
class C(A, B):
    def __init__(self):
        A.__init__(self)
        B.__init__(self)

    def getName(self):
        return self.name

C1 = C()
print(C1.getName())
```

Resolving the Conflicts with python multiple inheritance

Class C inherits both A and B. And both of them has an attribute ‘name’. From which class the value of name will be inherited in C?

- The name when printed is ‘**Richard**’ instead of ‘John’. Let’s try to understand what’s happening here. In the constructor of C, the first constructor called is the one of A.
- So, the value of name in C becomes same as the value of name in A.
- But after that, when the constructor of B is called, the value of name in C is overwritten by the value of name in B

The hierarchy becomes completely depended on the order of __init__() calls inside the subclass.

To deal with it perfectly, there is a protocol named **MRO (Method Resolution Order).**

What is MRO?

In the case of multiple inheritance a given attribute is first searched in the current class if it’s not found then it’s searched in the parent classes.

The parent classes are searched in a depth-first, left-right fashion and each class is searched once.

```
class A:
    def __init__(self):
        super().__init__()
        self.name = 'John'
        self.age = 23

    def getName(self):
        return self.name


class B:
    def __init__(self):
        super().__init__()
        self.name = 'Richard'
        self.id = '32'

    def getName(self):
        return self.name


class C(A, B):
    def __init__(self):
        super().__init__()

    def getName(self):
        return self.name

C1 = C()
print(C1.getName())
```

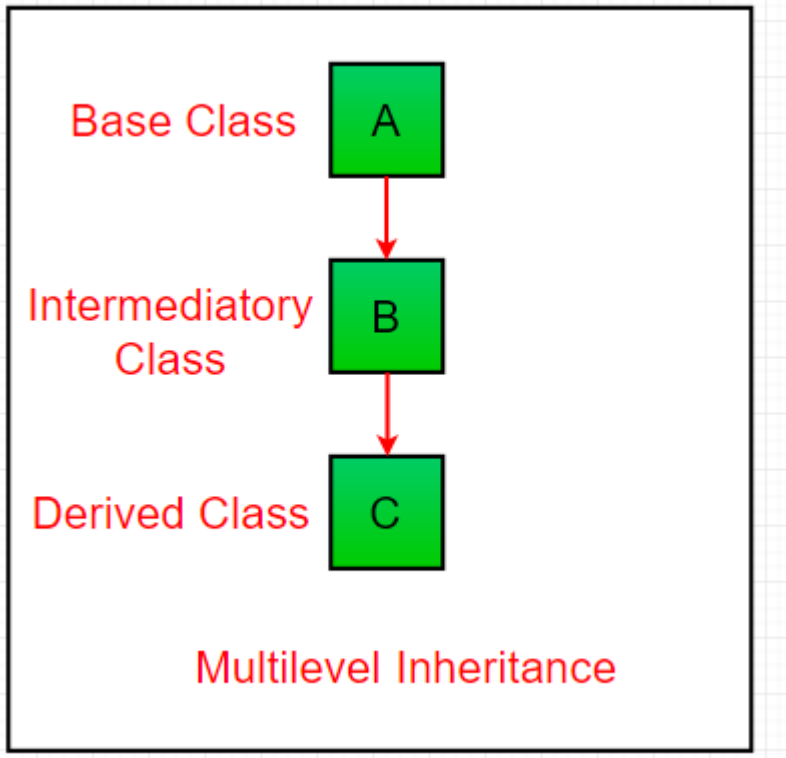
Once the constructor of A is called and attribute ‘name’ is accessed, it doesn’t access the attribute ‘name’ in B

OUTPUT

John

Multilevel Inheritance

In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to a relationship representing a child and grandfather.



Multilevel Inheritance

```
# Base class
class Grandfather:

    def __init__(self, grandfathername):
        self.grandfathername = grandfathername

# Intermediate class
class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername

        # invoking constructor of Grandfather class
        Grandfather.__init__(self, grandfathername)

# Derived class
class Son(Father):
    def __init__(self,sonname, fathername, grandfathername):
        self.sonname = sonname

        # invoking constructor of Father class
        Father.__init__(self, fathername, grandfathername)

    def print_names(self):
        print('Grandfather name :', self.grandfathername)
        print("Father name :", self.fathername)
        print("Son name :", self.sonname)
```

```
s1 = Son('Prince', 'Rampal', 'Lal mani')

s1.print_names()
```

Multilevel Inheritance

super() function in Python:

Python super function provides us the facility to refer to the parent class explicitly. It is basically useful where we have to call superclass functions. It returns the proxy object that allows us to refer parent class by ‘super’.

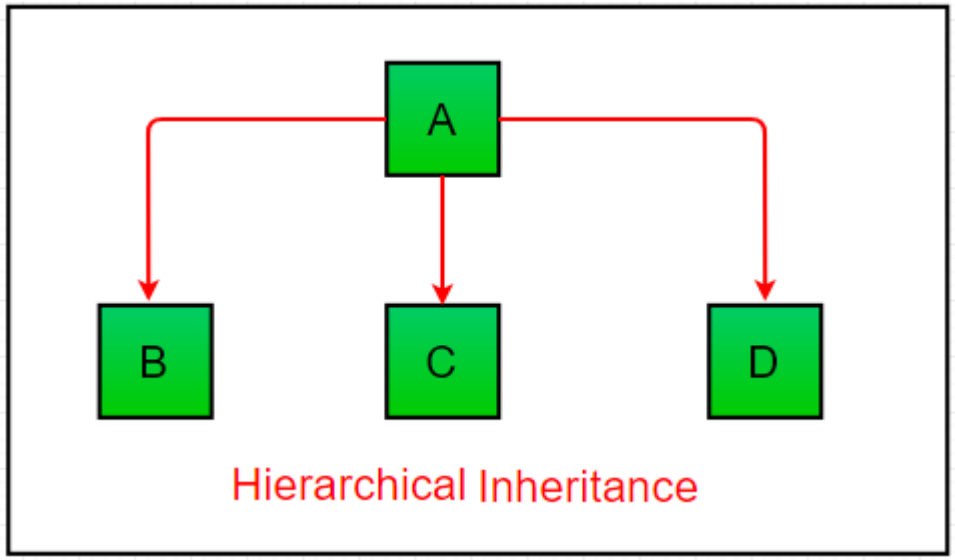
Example:

```
super().__init__()
```

Call parent class constructor from child constructor

Hierarchical Inheritance

When more than one derived classes are created from a single base this type of inheritance is called hierarchical inheritance.



Hierarchical Inheritance

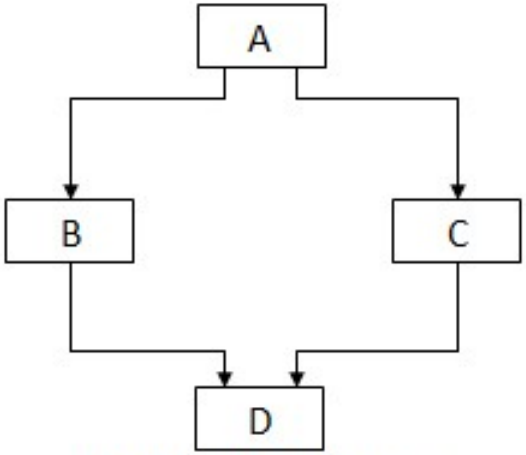
In this program, we have a parent (base) class and two child (derived) classes.

```
# Base class
class Parent:
 def func1(self):
  print("This function is in parent class.")

# Derived class1
class Child1(Parent):
 def func2(self):
  print("This function is in child 1.")

# Derivied class2
class Child2(Parent):
 def func3(self):
  print("This function is in child 2.")
```

```
# Driver's code
object1 = Child1()
object2 = Child2()
object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Hybrid Inheritance

Inheritance consisting of multiple types of inheritance is called hybrid inheritance.



Hybrid Inheritance

Hybrid Inheritance

```
class School:
 def func1(self):
  print("This function is in school.")

class Student1(School):
 def func2(self):
  print("This function is in student 1. ")

class Student2(School):
 def func3(self):
  print("This function is in student 2.")

class Student3(Student1, School):
 def func4(self):
  print("This function is in student 3.")
```

```
# Driver's code
object = Student3()
object.func1()
object.func2()
```

Hybrid Inheritance

```
class A(object):
  def __init__(self):
    super().__init__()
    print ("function of class A")
class B(A):
  def __init__(self):
    super().__init__()
    print ("function of class B")

class C(A):
  def __init__(self):
    super().__init__()
    print ("function of class C")

class D(B,C):
  def __init__(self):
    super().__init__()
    print ("function of class D")

j = D()
```

OUTPUT

```
function of class A
function of class C
function of class B
function of class D
```

use **super()** method.

Only one copy of class A Methods will be available in Class D